

**TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
A DISTANCIA — UTN**

Programación II

Trabajo Práctico Integrador

Food Store – Sistema de Gestión de Pedidos de Comida

Grupo: G41

Integrantes:

Ezequiel Nieva

Tomás Ghiglione

Fecha de entrega: 21/6/2026

Contenido

1. Marco Teórico	3
1.1 Java 21	3
1.2 Programación Orientada a Objetos.....	3
1.3 Colecciones	3
1.4 Manejo de Excepciones.....	3
2. Decisiones Técnicas y Arquitectura	4
2.1 Estructura de paquetes.....	4
2.2 Separación de responsabilidades (capas).....	4
2.3 Soft Delete.....	4
2.4 Validación de entradas	4
3. Descripción del Sistema	5
3.1 Modelo de dominio	5
3.2 Funcionalidades implementadas	5
Gestión de Categorías.....	5
Gestión de Productos	5
Gestión de Usuarios.....	5
Gestión de Pedidos	5
4. Dificultades y Soluciones	7
4.1 Sincronización entre Categoría y Producto.....	7
4.2 Lectura de enteros con Scanner	7
4.3 Validación de valores negativos.....	7
5. Bibliografía / Webgrafía	8
6. Links del Proyecto	8

1. Marco Teórico

El presente proyecto fue desarrollado en Java 21 como aplicación de consola, aplicando los principios de la Programación Orientada a Objetos (POO). A continuación, se describen las tecnologías y conceptos clave utilizados.

1.1 Java 21

Java es un lenguaje de programación orientado a objetos, fuertemente tipado y de propósito general. En este proyecto se utilizó Java 21 (LTS), aprovechando características modernas como switch expressions con arrow syntax y var implícito. El entorno de desarrollo utilizado fue NetBeans IDE con Apache Ant como sistema de build.

1.2 Programación Orientada a Objetos

Los principales conceptos de POO aplicados en el proyecto son:

- Herencia: todas las entidades del sistema (Categoría, Producto, Usuario, Pedido, DetallePedido) heredan de una clase abstracta base (Base), que centraliza los atributos comunes id, eliminado y createdAt.
- Encapsulamiento: todos los atributos de las entidades son privados, accedidos únicamente a través de getters y setters.
- Polimorfismo: la interfaz Calculable define el comportamiento calcularTotal(), implementado por la clase Pedido con su propia lógica.
- Abstracción: la clase Base es abstracta y no puede instanciarse directamente.

1.3 Colecciones

Como el sistema no utiliza base de datos, toda la información se almacena en memoria durante la ejecución del programa mediante el uso de `ArrayList<T>`. Cada service mantiene su propia colección de objetos y un contador de ids autoincrementales.

1.4 Manejo de Excepciones

Se crearon tres excepciones propias que extienden de `Exception`:

- `EntidadNoEncontradaException`: lanzada cuando se busca una entidad por id y no existe.
- `StockInvalidoException`: lanzada cuando se intenta agregar un detalle de pedido con cantidad mayor al stock disponible.
- `ValidacionException`: lanzada cuando falla una validación de campos (nombre vacío, mail duplicado, precio negativo, etc.).

2. Decisiones Técnicas y Arquitectura

2.1 Estructura de paquetes

El proyecto se organizó en los siguientes paquetes:

- `integradoR.prog2` — Main y `MenuPrincipal` (punto de entrada e interacción con el usuario).
- `integradoR.prog2.entities` — Clases del modelo de dominio (`Base`, `Categoría`, `Producto`, `Usuario`, `Pedido`, `DetallePedido`).
- `integradoR.prog2.enums` — Enumeraciones (`Rol`, `Estado`, `FormaPago`).
- `integradoR.prog2.exception` — Excepciones propias del sistema.
- `integradoR.prog2.interfaces` — Interfaz `Calculable`.
- `integradoR.prog2.service` — Lógica de negocio y gestión de colecciones (`CategoriaService`, `ProductoService`, `UsuarioService`, `PedidoService`).

2.2 Separación de responsabilidades (capas)

Se adoptó una arquitectura de tres capas: la capa de entidades contiene el modelo de dominio puro sin lógica de negocio; la capa de servicios encapsula las validaciones, reglas de negocio y gestión de las colecciones; y la capa de vista (`MenuPrincipal`) se encarga únicamente de la interacción con el usuario a través de la consola, sin contener reglas de negocio.

2.3 Soft Delete

Se implementó baja lógica en todas las entidades: en lugar de remover físicamente los objetos de las colecciones, se marca el campo `eliminado = true`. Esto permite mantener el historial y evitar inconsistencias, por ejemplo, en pedidos que referencian productos o usuarios eliminados.

2.4 Validación de entradas

El menú utiliza métodos auxiliares (`leerEntero`, `leerEnteroPositivo`, `leerEnteroNoNegativo`, `leerDoubleNoNegativo`) que validan el tipo y rango del valor ingresado, repitiendo la solicitud hasta recibir un valor válido sin interrumpir el flujo del programa.

3. Descripción del Sistema

Food Store es un sistema de gestión de pedidos de comida desarrollado como aplicación de consola en Java. Permite realizar operaciones CRUD completas sobre cuatro entidades principales: Categorías, Productos, Usuarios y Pedidos.

3.1 Modelo de dominio

El sistema cuenta con las siguientes entidades, todas heredando de la clase abstracta Base:

- Categoría: agrupa productos. Atributos: nombre, descripción.
- Producto: ítem del catálogo. Atributos: nombre, precio, descripción, stock, imagen, disponible. Se asocia a una Categoría (N:1).
- Usuario: operador del sistema. Atributos: nombre, apellido, mail (único), celular, contraseña, rol (ADMIN/USUARIO).
- Pedido: compra realizada por un usuario. Atributos: fecha, estado, total, forma Pago. Composición 1:N con DetallePedido. Implementa la interfaz Calculable.
- DetallePedido: línea de un pedido. Atributos: cantidad, subtotal. Se asocia a un Producto (N:1).

3.2 Funcionalidades implementadas

Gestión de Categorías

- Listar categorías activas.
- Crear categoría (con validación de nombre único y campos no vacíos).
- Editar nombre y descripción.
- Eliminar lógicamente (soft delete).

Gestión de Productos

- Listar productos activos con su categoría asociada.
- Crear producto (precio ≥ 0 , stock ≥ 0 , categoría válida).
- Editar todos los campos incluyendo la categoría.
- Eliminar lógicamente.

Gestión de Usuarios

- Listar usuarios activos.
- Crear usuario (mail único, campos no vacíos, rol seleccionable).
- Editar datos del usuario con revalidación de mail.
- Eliminar lógicamente.

Gestión de Pedidos

- Listar pedidos activos con usuario, estado, forma de pago y total.
- Crear pedido: selección de usuario y forma de pago, carga de N detalles usando addDetallePedido(), cálculo del total mediante la interfaz Calculable.

- Actualizar estado (PENDIENTE/CONFIRMADO/TERMINADO/CANCELADO) y forma de pago.
- Eliminar lógicamente.
- Validación de stock al agregar detalles: lanza `StockInvalidoException` si la cantidad supera el stock disponible.

4. Dificultades y Soluciones

4.1 Sincronización entre Categoría y Producto

Al crear un producto y asociarlo a una categoría, inicialmente la lista interna de productos de la categoría no se actualizaba. Esto se debía a que solo se seteaba la referencia en el producto (`producto.setCategoria(categoría)`) pero no se actualizaba la colección inversa. La solución fue agregar al método `crear()` del `ProductoService` la línea `producto.getCategoria().getProductos().add(producto)`, manteniendo así la bidireccionalidad de la relación. Esto se detectó al realizar las pruebas luego de finalizar el menú.

4.2 Lectura de enteros con Scanner

El uso de `scanner.nextInt()` generaba problemas al dejar el salto de línea en el buffer, causando lecturas incorrectas en el siguiente campo de texto. La solución fue utilizar siempre `scanner.nextLine()` y parsear el valor con `Integer.parseInt()`, encapsulado en un método auxiliar `leerEntero()` que retorna -1 en caso de error, evitando excepciones no controladas.

4.3 Validación de valores negativos

En la versión inicial, si el usuario ingresaba un precio o stock negativo el sistema mostraba el error y volvía al menú, obligando al usuario a navegar nuevamente hasta el formulario. Se implementaron métodos auxiliares con loop de reintento (`leerEnteroPositivo`, `leerEnteroNoNegativo`, `leerDoubleNoNegativo`) que repiten la solicitud hasta recibir un valor válido, mejorando significativamente la experiencia de uso tanto para nosotros al realizar pruebas como a un potencial usuario para no perder la carga por un error de tipeo.

5. Bibliografía / Webgrafía

- Oracle. (2024). Java Collections Framework. <https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>
- Material de cátedra — Programación II, UTN Tecnicatura Universitaria en Programación a Distancia.
- Baeldung. (2024). Guide to Java 21. <https://www.baeldung.com/java-21-new-features>

6. Links del Proyecto

- Repositorio GitHub: https://github.com/tghiglione/UTN-TUPaDProgramacion2_G41
- Video demostrativo: [Link Video Demostrativo](#)
- Documentación PDF: Este mismo informe.